

Self Study Report — Day10

1. Parallel Programming and Concurrency

Concurrency means multiple tasks are *in progress* during overlapping time periods — they don't necessarily run at the exact same instant, they just take turns making progress (e.g. a single CPU core switching rapidly between tasks). **Parallelism** means multiple tasks are *literally executing at the same instant*, which requires multiple CPU cores.

Concurrency is about **dealing with** many things at once.
Parallelism is about **doing** many things at once.

In .NET/C#, the main tools for this are:

- **Task Parallel Library (TPL)** — `System.Threading.Tasks.Task` represents a unit of work that can run on a thread-pool thread.

```
Task t1 = Task.Run(() => DoWork1());  
Task t2 = Task.Run(() => DoWork2());  
Task.WaitAll(t1, t2);
```

- **Parallel.For / Parallel.ForEach** — data parallelism: splitting a loop's iterations across multiple threads automatically.

```
Parallel.For(0, 1000, i => ProcessItem(i));
```

- **PLINQ (Parallel LINQ)** — `.AsParallel()` on a LINQ query, letting the runtime split the work across cores.
- **Thread synchronization primitives** — `lock`, `Monitor`, `Mutex`, `SemaphoreSlim`, `Interlocked` — needed whenever parallel tasks share mutable state, to avoid race conditions.

Why it matters: modern CPUs have many cores, and a program that only ever uses one core wastes most of the machine's capacity for CPU-bound work (image processing, calculations, data transformations). The trade-off is complexity: shared state accessed from multiple threads at once introduces race conditions, deadlocks, and bugs that are much harder to reproduce and debug than single-threaded code.

2. Unit Testing and Test-Driven Development (TDD)

Unit testing means writing small, automated tests that each verify one specific piece of behavior in isolation (a single method or class), separate from the rest of the system. In C#, this is typically done with a test framework like **xUnit**, **NUnit**, or **MSTest**.

```
[Fact] // xUnit  
public void Add_TwoPositiveNumbers_ReturnsSum()  
{  
    var calculator = new Calculator();  
    int result = calculator.Add(2, 3);  
    Assert.Equal(5, result);  
}
```

A good unit test follows the **Arrange-Act-Assert** pattern: set up the inputs, call the method being tested, then check the result.

Test-Driven Development (TDD) is a development *workflow* built around unit tests, following the **Red → Green → Refactor** cycle:

1. **Red** — write a failing test for a feature that doesn't exist yet.
2. **Green** — write the *minimum* code needed to make that test pass.
3. **Refactor** — clean up the implementation (and/or the test) while keeping all tests passing.

Why it matters: - Tests act as a safety net — a refactor or a new feature that breaks existing behavior is caught immediately, not discovered in production. - TDD forces you to think about a method's contract (inputs/outputs/edge cases) *before* writing the implementation, which tends to produce simpler, more focused designs. - Well-tested code is easier to change with confidence, which matters a lot in real business codebases that evolve for years.

3. Asynchronous Programming with `async` and `await`

Asynchronous programming lets a program **start** a long-running operation (a network call, a database query, a file read) **without blocking the calling thread** while waiting for it to finish. The thread is freed up to do other work, and resumes exactly where it left off once the operation completes.

In C#, this is built around `Task` / `Task<T>` and two keywords:

```
public async Task<string> GetDataAsync()
{
    HttpClient client = new HttpClient();
    string result = await
client.GetStringAsync("https://example.com");
    return result;
}
```

- **async** marks a method as containing asynchronous operations — it allows the use of `await` inside it, and changes how the method's return value works (its real return type is wrapped in a `Task/Task<T>`).
- **await** suspends execution of the method at that point *without blocking the thread*, until the awaited `Task` completes. Once it completes, execution resumes on that point (via a continuation), usually back on an appropriate context.

Why not just use regular (synchronous) calls? A synchronous call to `client.GetString(...)` would block the calling thread entirely until the network response arrives — wasted time the thread could have spent doing other work. This matters enormously for: - **UI applications** — a blocking call on the UI thread freezes the whole interface until it returns. - **Servers/APIs** — a thread blocked waiting on I/O can't serve other requests, so `async` I/O lets one server handle far more concurrent requests with the same number of threads.

Important distinction: `async/await` is primarily about **I/O-bound** concurrency (waiting on something external — network, disk, database), not about parallel *computation*. For CPU-bound parallel work, TPL constructs like `Task.Run`, `Parallel.For`, or `PLINQ` are the right tool instead.

4. What Is Asynchronous Programming

Asynchronous programming, more broadly (beyond just C#'s keywords), is a programming model where an operation is *initiated* and the program continues running other code instead of waiting idle for that operation to finish. When the operation eventually completes, the program is notified (via a callback, a Promise/Task, an event, or — in C# — the await continuation) and can process the result at that point.

Core idea: “**don’t block waiting — go do something else, and come back when it’s ready.**”

This differs from: - **Synchronous programming** — each statement fully completes before the next one starts; the calling thread sits idle during any wait. - **Multithreading/parallelism** — asynchronous programming does *not* necessarily mean multiple threads are running. A single thread can juggle many pending asynchronous operations (this is exactly what happens with async/await in a typical I/O scenario — no extra thread is spun up just to “wait”; the thread is released back to the thread pool while the external operation is in progress).

Real-world analogy: ordering food at a restaurant and getting a buzzer instead of standing at the counter until your order is ready. You go sit down, talk to your friends, do other things — the buzzer (the “callback”) notifies you the moment your food (the “result”) is ready. You were never blocked waiting; you were just... waiting differently, without being stuck.

Why it’s essential in modern software: - Keeps applications responsive (UI never freezes). - Lets servers handle a large number of simultaneous requests efficiently, without needing one dedicated thread per request. - Matches how most real-world work actually behaves: most of the *time* spent waiting on a network call, disk read, or database query is time the CPU could otherwise be doing useful work — asynchronous programming lets it do exactly that.